Análise de Dados com OpenMP

Coeficiente de variação de estatura e peso — dados agrupados em classes

Relatório do Exercício Final

1. Introdução

O programa **cv_openmp.c** lê um arquivo texto com a estatura (cm) e o peso (kg) de 1000 pessoas (uma pessoa por linha, valores separados por espaço), agrupa cada variável em classes (8 cm para estatura e 4 kg para peso) e calcula, para dados agrupados, a média ponderada μ, o desvio padrão populacional σ e o coeficiente de variação CV = σ/μ × 100%. Os laços principais foram paralelizados com OpenMP. O mesmo arquivo-fonte gera a versão sequencial (compilando sem `-fopenmp`, pois as diretivas `#pragma omp` são então ignoradas) e a paralela, garantindo uma comparação justa.

Observação sobre os dados: o enunciado não fornece o arquivo, então foi gerado um conjunto sintético de 1000 pessoas (`dados.txt`, semente fixa 42; estatura ≈ N(170; 9) e peso correlacionado com a estatura). Para usar os dados reais, basta trocar o arquivo; o programa se adapta (limites e número de classes são calculados a partir do mínimo e do máximo).

2. Metodologia

Classes. O limite inferior da 1ª classe é ⌊mínimo⌋ e o número de classes é k = ⌊(máximo − limite inferior)/h⌋ + 1, com h = 8 (estatura) ou h = 4 (peso). Cada observação x vai para a classe de índice ⌊(x − lo)/h⌋, e o ponto médio é x_i = lo + (i + 0,5)·h. Intervalos são fechados à esquerda e abertos à direita.

Fórmulas (as do enunciado): μ = Σ(x_i·f_i)/N; σ = √[Σ f_i·(x_i − μ)^2 / N]; CV = (σ/μ)·100%.

3. O que foi paralelizado e por quê

Etapa	Diretiva OpenMP	Por que
1. Mínimo e máximo dos 1000 valores (define as classes)	`parallel for reduction(min:…) reduction(max:…)`	Cada thread calcula o extremo da sua fatia e o OpenMP combina os resultados; sem escrita compartilhada.
2. Distribuição de frequências (histograma)	`parallel for reduction(+:fa[:ka], fp[:kp])`	É o laço de **maior custo**: percorre as N pessoas. Várias threads incrementariam o mesmo contador (condição de corrida); a redução sobre array dá a cada thread um histograma privado, somados no fim, sem critical/atomic. Estatura e peso são tratados no mesmo laço (uma só leitura dos dados).
3. Soma Σ x_i·f_i (média)	`parallel for reduction(+:sa)` (e sp)	Soma associativa sobre as classes; redução elimina a dependência.
4. Soma Σ $f_i(x_i-\mu)^2$ (desvio padrão)	`parallel for reduction(+:va)` (e vp)	Mesma estrutura da etapa 3; só pode começar depois da média (dependência de dados).

Não paralelizado: leitura do arquivo (I/O sequencial) e o cálculo final de σ e CV (poucas operações escalares). Há uma barreira implícita ao fim de cada `parallel for`, que garante que a média esteja pronta antes do desvio padrão.

Ressalva honesta: as etapas 3 e 4 percorrem apenas k classes (7 e 17), o que é minúsculo; paralelizá-las atende ao enunciado, mas o custo de criar a região paralela supera o ganho

nesse tamanho. O ganho potencial real está nas etapas 1 e 2, que dependem de N.

4. Resultados (N = 1000)

Estatura (classes de 8 cm)

Classe (cm)	Ponto médio x_i	f_i	$x_i \cdot f_i$
145 ├─ 153	149	30	4470
153 ├─ 161	157	140	21980
161 ├─ 169	165	287	47355
169 ├─ 177	173	342	59166
177 ├─ 185	181	155	28055
185 ├─ 193	189	42	7938
193 ├─ 201	197	4	788
Total		1000	169752

Peso (classes de 4 kg)

Classe (kg)	Ponto médio x_i	f_i	$x_i \cdot f_i$
42 ├─ 46	44	32	1408
46 ├─ 50	48	30	1440
50 ├─ 54	52	43	2236
54 ├─ 58	56	64	3584
58 ├─ 62	60	107	6420
62 ├─ 66	64	107	6848
66 ├─ 70	68	133	9044
70 ├─ 74	72	136	9792
74 ├─ 78	76	128	9728
78 ├─ 82	80	93	7440
82 ├─ 86	84	57	4788
86 ├─ 90	88	30	2640
90 ├─ 94	92	15	1380
94 ├─ 98	96	14	1344
98 ├─ 102	100	7	700
102 ├─ 106	104	2	208
106 ├─ 110	108	2	216
Total		1000	69216

Variável	Média μ	Desvio padrão σ	CV
Estatura	169,7520 cm	9,1605 cm	**5,3964 %**
Peso	69,2160 kg	11,8602 kg	**17,1351 %**

As versões sequencial e paralela produziram resultados idênticos. Como verificação, o CV calculado sobre os dados brutos (sem agrupar) é 5,22 % (estatura) e 17,25 % (peso): o agrupamento introduz uma pequena aproximação, pois cada pessoa é representada pelo ponto médio da sua classe, algo esperado nesse método. Em ambos os casos o peso varia muito mais, em termos relativos, que a estatura (CV ≈ 3× maior).

5. Análise de desempenho

Ambiente: AMD Ryzen 5 5500 (6 núcleos físicos, 12 threads lógicas), Windows, compilador clang 23.1.5 (MSYS2 CLANG64), `-O2 -fopenmp` (paralela) e `-O2` (sequencial). Os tempos foram medidos pelo próprio programa (`QueryPerformanceCounter`), apenas no cálculo, sem a leitura do arquivo. Cada configuração foi medida 5 vezes e o valor reportado é a **mediana**; dentro de cada execução o cálculo foi repetido (20000, 20 e 5 vezes, conforme N) e o tempo é a média por repetição.

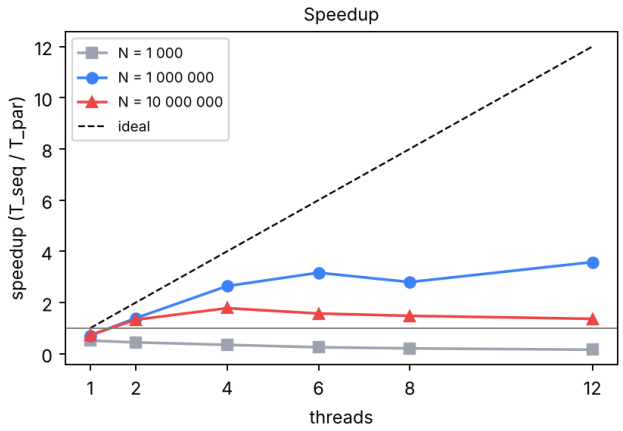
Como 1000 pessoas é um volume muito pequeno para medir paralelismo, o conjunto foi também replicado 1000× e 10000× ($N = 10^6$ e 10^7) apenas para o benchmark; a replicação multiplica todas as frequências e não altera μ, σ nem o CV. O speedup é $S = T_{seq}/T_{par}$ e a eficiência é $E = S/p$, com p o número de threads.

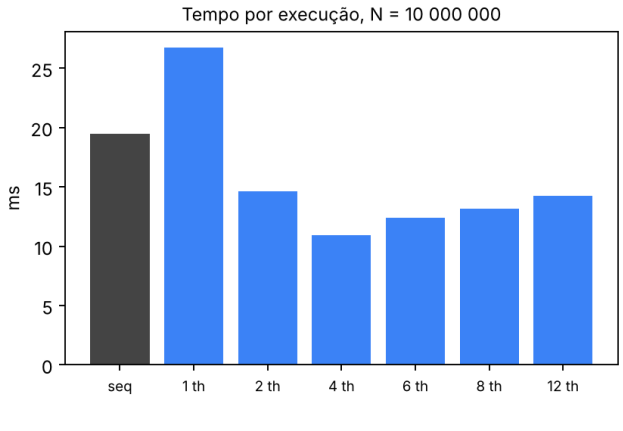
Tabela 1 — Tempo médio por execução (ms)

N	Seq. (ms)	1 th	2 th	4 th	6 th	8 th	12 th
1 000	0,0018	0,0034	0,0040	0,0051	0,0070	0,0085	0,0112
1 000 000	1,8674	2,6139	1,3424	0,7066	0,5901	0,6674	0,5217
10 000 000	19,4679	26,7720	14,6562	10,9341	12,3927	13,1524	14,2719

Tabela 2 — Speedup / eficiência

N	1 th	2 th	4 th	6 th	8 th	12 th
1 000	0,51× / 51%	0,44× / 22%	0,35× / 9%	0,25× / 4%	0,21× / 3%	0,16× / 1%
1 000 000	0,71× / 71%	1,39× / 70%	2,64× / 66%	3,16× / 53%	2,80× / 35%	3,58× / 30%
10 000 000	0,73× / 73%	1,33× / 66%	1,78× / 45%	1,57× / 26%	1,48× / 19%	1,36× / 11%





Interpretação.

• **N = 1000 (o problema do enunciado):** a versão sequencial leva cerca de 1,8 μs por execução e a paralela é sempre mais lenta, e piora conforme aumentam as threads (speedup de 0,16× com 12 threads). O custo de criar e sincronizar a equipe de threads é muito maior que o próprio cálculo. Para esse tamanho, **paralelizar não compensa**.

• **$N = 10^6$:** o paralelismo passa a compensar. O speedup chega a 2,64× com 4 threads, 3,16× com 6 e 3,58× com 12. A eficiência cai de forma gradual (≈ 70% com 2 threads, ≈ 53% com 6), o que é esperado: parte do trabalho (histograma privado por thread, redução, criação da região paralela) não diminui com mais threads.

• **N = 10^7:** o melhor resultado foi 1,78× com 4 threads; com mais threads o desempenho piora (1,36× com 12). O laço dominante só faz uma leitura sequencial de dois vetores grandes (≈ 160 MB) e poucas operações por elemento, então o limite provável é a **largura de banda da memória**, e não o número de núcleos: a partir de certo ponto, mais threads disputam o mesmo barramento e passam a atrapalhar. Os 12 "núcleos" lógicos do Ryzen são 6 físicos com SMT, o que também ajuda a explicar a perda além de 6 threads. Essas causas são hipóteses coerentes com os dados; não foram isoladas por medições adicionais (por exemplo, contadores de hardware).

• **Paralela com 1 thread mais lenta que a sequencial** (speedup ≈ 0,5 a 0,7): é o custo fixo do OpenMP (runtime, histograma privado e redução) mesmo sem concorrência. Por isso o ganho só aparece com 2 threads ou mais.

• **Variações pontuais** (por exemplo, 12 threads melhor que 8 em N = 10^6) estão dentro do ruído esperado de medições na escala de décimos de milissegundo em um computador de uso geral; a tendência geral é o que importa.

6. Conclusão

O coeficiente de variação obtido foi **5,40 % para a estatura** e **17,14 % para o peso**. A paralelização com reductions é correta (resultados idênticos ao da versão sequencial, sem condições de corrida). Quanto ao desempenho: para os 1000 registros do enunciado a versão paralela é mais lenta que a sequencial, pois o problema é pequeno demais; para volumes maiores houve ganho real, de até ≈ 3,6× (N = 10^6, 12 threads) e ≈ 1,8× (N = 10^7, 4 threads), limitado pela eficiência decrescente e, para N maior, provavelmente pela largura de banda da memória.

Arquivos: cv_openmp.c (código), dados.txt (entrada), bench.ps1 e bench.csv (benchmark). **Compilar e executar (MSYS2 CLANG64):** `clang -O2 -fopenmp cv_openmp.c -o cv_par -lm` e `./cv_par.exe dados.txt`.

Apêndice — Código-fonte (trecho paralelizado)

```
static void calcular(const double *alt, const double *peso, long N,
                     Dist *da, Dist *dp)
{
    double amin = alt[0],  amax = alt[0];
    double pmin = peso[0], pmax = peso[0];

    /* (1) Mínimos e máximos: redução paralela */
    #pragma omp parallel for reduction(min:amin,pmin) reduction(max:amax,pmax)
    for (long i = 0; i < N; i++) {
        if (alt[i]  < amin) amin = alt[i];
        if (alt[i]  > amax) amax = alt[i];
        if (peso[i] < pmin) pmin = peso[i];
        if (peso[i] > pmax) pmax = peso[i];
    }

    da->h = H_ALT;  da->lo = floor(amin);
    dp->h = H_PESO; dp->lo = floor(pmin);
    da->k = (int)floor((amax - da->lo) / da->h) + 1;
    dp->k = (int)floor((pmax - dp->lo) / dp->h) + 1;

    long *fa = calloc(da->k, sizeof(long));
    long *fp = calloc(dp->k, sizeof(long));
    const int ka = da->k, kp = dp->k;
    const double loa = da->lo, ha = da->h, lop = dp->lo, hp = dp->h;

    /* (2) Distribuição de frequências: cada thread acumula num histograma
     *     privado (reduction sobre array) -> sem condição de corrida.   */
    #pragma omp parallel for reduction(+:fa[:ka], fp[:kp])
    for (long i = 0; i < N; i++) {
        fa[(int)((alt[i]  - loa) / ha)]++;
        fp[(int)((peso[i] - lop) / hp)]++;
    }
    da->freq = fa; dp->freq = fp;

    /* (3) Média ponderada: soma(xi*fi) / N */
    double sa = 0.0, sp = 0.0;
    #pragma omp parallel for reduction(+:sa)
    for (int i = 0; i < ka; i++) sa += (loa + (i + 0.5) * ha) * fa[i];
    #pragma omp parallel for reduction(+:sp)
    for (int i = 0; i < kp; i++) sp += (lop + (i + 0.5) * hp) * fp[i];
    da->media = sa / N;
    dp->media = sp / N;

    /* (4) Desvio padrão: soma(fi*(xi-mu)^2) / N */
    double va = 0.0, vp = 0.0;
    const double ma = da->media, mp = dp->media;
    #pragma omp parallel for reduction(+:va)
    for (int i = 0; i < ka; i++) {
        double d = (loa + (i + 0.5) * ha) - ma;
        va += fa[i] * d * d;
    }
    #pragma omp parallel for reduction(+:vp)
    for (int i = 0; i < kp; i++) {
        double d = (lop + (i + 0.5) * hp) - mp;
        vp += fp[i] * d * d;
    }
    da->desvio = sqrt(va / N);
    dp->desvio = sqrt(vp / N);

    /* (5) Coeficiente de variação */
    da->cv = da->desvio / da->media * 100.0;
    dp->cv = dp->desvio / dp->media * 100.0;
}
```